

Evaluating Tool-Selection Accuracy in Multi-Tool Agentic AI Systems

Abstract

Multi-tool agentic artificial intelligence systems increasingly combine large language models (LLMs) with external APIs, databases, search engines, code interpreters, browsers, enterprise services, and other executable capabilities. In such systems, successful task completion depends not only on the model's ability to reason or generate valid arguments, but also on selecting an appropriate tool from a potentially large and heterogeneous tool set. Tool-selection accuracy is therefore a distinct and consequential component of agent reliability. This paper reviews the emerging literature on tool-augmented LLMs and proposes a conceptual framework for evaluating tool-selection accuracy in multi-tool agentic systems. The review synthesizes foundational approaches including ReAct, Toolformer, Gorilla, API-Bank, ToolLLM/ToolBench, ToolQA, AgentBench, WebArena, τ -bench, and ToolSandbox, together with recent work on tool overuse, self-correction, live API evaluation, and deployment-oriented agent evaluation. The analysis distinguishes tool-selection accuracy from related capabilities such as parameter generation, execution correctness, planning, and end-to-end task success. It argues that existing benchmarks frequently conflate these dimensions, making it difficult to determine whether an agent failed because it selected the wrong tool, generated invalid arguments, followed an incorrect multi-step plan, or misinterpreted tool output. A multidimensional evaluation protocol is consequently proposed, incorporating top-k selection accuracy, necessity/abstention accuracy, tool-set scaling, semantic equivalence, argument validity, sequence-level selection accuracy, cost-aware utility, robustness to tool-schema changes, and consistency across repeated trials. The paper identifies major research gaps involving dynamic tool ecosystems, near-duplicate tools, tool abstention, uncertainty calibration, adversarial tool descriptions, stateful dependencies, and standardized causal attribution of agent failures. Future research should treat tool selection as a decision problem under uncertainty rather than merely as function-call generation, while developing reproducible benchmarks that evaluate both correctness and operational efficiency.

Keywords: agentic AI, large language models, tool use, tool selection, function calling, API selection, autonomous agents, evaluation, benchmarks, tool-augmented LLMs

1. Introduction

Large language models have evolved from systems that primarily generate text into components of agentic architectures capable of interacting with external environments. Contemporary agents may search the web, query databases, invoke APIs, execute code, manipulate documents, browse websites, access enterprise systems, or coordinate several specialized tools within a single task. The resulting architecture changes the nature of reliability: the model is no longer required merely to produce an appropriate textual answer, but must determine **whether external action is necessary, which tool should be used, when it should be invoked, what arguments it should receive, and how its output should affect subsequent decisions.**

Early work established the conceptual foundation for this paradigm. ReAct demonstrated that reasoning and action can be interleaved so that an LLM reasons about a task, performs an external action, observes its result, and updates its subsequent behavior (Yao et al., 2023). Toolformer subsequently demonstrated

that language models can learn when and how to invoke APIs, explicitly framing tool use as a learned decision concerning API selection, arguments, and incorporation of returned information (Schick et al., 2023).

As tool inventories have expanded, however, the problem has become substantially more difficult. ToolLLM, for example, constructed ToolBench from 16,464 real-world APIs spanning 49 categories and explicitly included both single-tool and multi-tool scenarios (Qin et al., 2024). Gorilla similarly emphasized the difficulty of selecting among large numbers of overlapping and changing APIs and showed that retrieval can help models adapt to updated API documentation (Patil et al., 2024).

This development motivates a central research question:

How accurately can an agent select the appropriate tool, or sequence of tools, from a heterogeneous and potentially dynamic tool set, and how should that accuracy be measured independently of other agent capabilities?

The question is important because conventional end-to-end success metrics can obscure the source of failure. An agent can select the correct API but provide an invalid argument; conversely, it can select a semantically equivalent alternative tool and still complete the task successfully. A benchmark reporting only final task success therefore cannot determine whether improvements arise from better tool retrieval, better reasoning, better argument generation, improved error recovery, or some combination.

This paper reviews the literature surrounding this problem and develops a structured evaluation perspective. It makes four contributions. First, it distinguishes tool-selection accuracy from adjacent dimensions of tool use. Second, it synthesizes current benchmark designs and identifies what each does—and does not—measure. Third, it proposes a multidimensional framework for evaluating tool-selection accuracy in realistic multi-tool environments. Fourth, it identifies research gaps concerning dynamic tool ecosystems, tool redundancy, abstention, uncertainty, statefulness, robustness, and causal failure attribution.

2. Background and Related Work

2.1 Tool-Augmented Language Models

Tool use extends an LLM's effective computational and informational capabilities beyond its parametric knowledge. Toolformer introduced a self-supervised approach in which a language model learns to insert API calls into text generation, with training covering decisions about whether to invoke a tool, which API to use, and how to incorporate its response (Schick et al., 2023).

ReAct instead framed tool use as part of an iterative reasoning-and-acting process. Its results on question answering, fact verification, ALFWorld, and WebShop showed that external interaction can reduce hallucination and support adaptive planning (Yao et al., 2023).

These systems established an important distinction between **generation** and **interaction**. A generated answer can be judged directly, whereas an action must be evaluated relative to an external state and task

objective. This distinction becomes especially important when an agent has access to multiple tools with overlapping capabilities.

2.2 API Selection and Tool Retrieval

API-Bank was among the first dedicated benchmarks to explicitly evaluate tool-augmented LLMs. It included 73 executable APIs, 314 annotated dialogues, and 753 API calls for evaluating planning, API retrieval, and API calling. Its broader training collection contained 2,138 APIs across 1,000 domains (Li et al., 2023).

ToolLLM/ToolBench substantially expanded the scale of the problem. Qin et al. (2024) collected 16,464 real-world APIs and created instructions involving single- and multi-tool tasks. ToolLLM incorporated a neural API retriever and a decision-tree-based reasoning method, illustrating an architecture in which **tool retrieval precedes or supports downstream tool execution**.

Gorilla addressed a closely related problem involving massive and frequently changing API collections. Its retrieval-aware architecture was designed to reduce API hallucination and adapt to test-time documentation changes, emphasizing that tool selection cannot be treated as a static classification problem when API ecosystems evolve (Patil et al., 2024).

2.3 Tool-Use and Agent Benchmarks

ToolQA evaluated LLM question answering with 13 external tools and deliberately attempted to distinguish questions requiring external tools from those solvable through internal knowledge (Zhuang et al., 2023). This distinction is fundamental to evaluating **tool necessity** rather than simply measuring whether a model can call an available tool.

AgentBench broadened evaluation from isolated API calls to agents operating in multiple environments. It evaluated LLMs across eight environments and emphasized reasoning and decision making under interaction rather than static question answering (Liu et al., 2024).

WebArena further increased ecological validity by providing realistic, self-hosted websites and long-horizon tasks. Its experiments reported an end-to-end success rate of 14.41% for the best GPT-4-based agent compared with 78.24% human performance, illustrating the difficulty of translating language-model competence into reliable environmental action (Zhou et al., 2024).

AgentBoard criticized the exclusive use of final success rates and introduced a fine-grained progress metric for multi-turn agent evaluation, thereby supporting more diagnostic analysis of where an agent fails during a trajectory (Ma et al., 2024).

3. What Is Tool-Selection Accuracy?

Tool-selection accuracy should be treated as a distinct construct rather than a synonym for tool-use success.

Consider a task x , a set of available tools $T = \{t_1, \dots, t_n\}$, and a ground-truth set of tools $T^*(x)$ that can validly contribute to solving the task. An agent selects t_i . At the simplest level:

$$Acc_{select} = \frac{1}{N} \sum_{j=1}^N 1[t_j \in T^*(x_j)].$$

This formulation is deliberately broader than exact-match evaluation. In realistic environments, multiple tools may be semantically valid. For example, two weather APIs may both satisfy a request for the current temperature. Treating only one API as correct would incorrectly penalize legitimate alternative solutions.

A useful evaluation hierarchy therefore contains at least six dimensions:

1. **Necessity detection:** Does the agent correctly determine whether external tool use is required?
2. **Tool identification:** Does it select a tool capable of satisfying the immediate objective?
3. **Tool-set discrimination:** Can it distinguish between similar or overlapping tools?
4. **Argument validity:** Does it provide arguments satisfying the selected tool's schema and semantic constraints?
5. **Execution correctness:** Does the tool call produce the desired state or information?
6. **Sequence correctness:** In multi-step tasks, does the agent select an appropriate sequence of tools?

These dimensions are related but not interchangeable.

For example, suppose an agent is asked to obtain a currency conversion. It may:

- correctly decide that a tool is needed;
- choose an exchange-rate API;
- supply the wrong currency code;
- receive an error;
- retry with another API;
- eventually succeed.

An end-to-end metric would record failure or perhaps success depending on recovery. A selection-specific metric should record that the initial tool selection was correct but argument generation or execution handling was defective.

This distinction is essential for scientific diagnosis.

4. A Taxonomy of Tool-Selection Errors

4.1 Wrong-Tool Selection

The simplest failure occurs when the selected tool cannot accomplish the intended operation. This may result from inadequate semantic matching between the user request and tool description, poor retrieval, or reasoning errors.

4.2 Tool Overuse

An agent may invoke an external tool when its use is unnecessary. Tool overuse increases latency, cost, and opportunities for failure. SMART specifically investigates this phenomenon, reporting reductions in tool usage while maintaining or improving task performance (Qian et al., 2025).

Consequently, selection accuracy should include a **negative class**: the correct decision may be to select no tool.

4.3 Tool Underuse

The opposite problem occurs when a task requires fresh information, computation, or an external state change but the agent attempts to answer using parametric knowledge. ToolQA's motivation is directly relevant: evaluation should distinguish tasks that genuinely require external tools from those that do not (Zhuang et al., 2023).

4.4 Near-Duplicate Tool Confusion

Large tool ecosystems often contain APIs with overlapping functionality. Gorilla explicitly identifies the challenge of selecting from large, overlapping and changing API collections (Patil et al., 2024).

This creates a harder problem than ordinary multiclass classification. Tools may differ only in:

- geographic coverage;
- authentication requirements;
- freshness;
- supported parameter ranges;
- output format;
- latency;
- cost;
- rate limits;
- reliability;
- legal or policy constraints.

An evaluation that ignores these differences may report artificially high selection accuracy.

4.5 Sequence-Level Selection Failure

Many tasks require a chain of tools. ToolLLM explicitly includes multi-tool scenarios and uses search over possible reasoning traces to construct solutions (Qin et al., 2024).

For a sequence $S = (t_1, t_2, \dots, t_k)$, a strict sequence metric can be expressed as:

$$Acc_{seq} = 1[S \equiv S^*],$$

but exact sequence matching is often too restrictive because different valid sequences may produce the same outcome. More useful evaluation should therefore combine sequence similarity with **functional equivalence**.

4.6 State-Dependent Selection Failure

Tool choice may depend on previous tool outputs or environment state. ToolSandbox explicitly introduces stateful tool execution, implicit dependencies between tools, conversational interaction, and dynamic evaluation of intermediate and final milestones (Lu et al., 2025).

Thus, a tool can be correct at time t but inappropriate at time $t + 1$. Static test cases cannot adequately capture this behavior.

5. Current Approaches to Evaluating Tool Selection

5.1 Exact-Match Function Calling

The simplest evaluation checks whether a model emits the expected function name and valid JSON arguments. This approach is reproducible and inexpensive, but it assumes a single canonical answer and can conflate selection and parameter errors.

Its primary advantage is diagnostic precision. Its weakness is ecological validity: production environments rarely guarantee that only one tool is acceptable.

5.2 Retrieval-Based Evaluation

Tool retrieval systems first identify candidate tools and then allow the LLM to select or invoke one. ToolLLM explicitly integrates a neural API retriever, while Gorilla uses retrieval to connect models with changing API documentation (Qin et al., 2024; Patil et al., 2024).

Retrieval metrics such as Recall@k are useful:

$$\text{Recall@}k = \frac{\text{\#tasks with a valid tool in top-}k}{N}.$$

However, retrieval recall alone does not measure whether the agent subsequently chooses the correct tool from the candidate set.

5.3 End-to-End Execution Evaluation

Benchmarks such as WebArena evaluate whether the final environment state satisfies a task objective rather than whether the generated action sequence matches a reference sequence. This is valuable because multiple action paths can legitimately solve the same task (Zhou et al., 2024).

The limitation is attribution. If an agent fails, the benchmark may not reveal whether the root cause was tool selection, planning, argument construction, observation interpretation, or recovery.

5.4 Stateful and Conversational Evaluation

τ -bench evaluates tool-agent-user interaction in realistic domains and introduces the pass^k metric to measure reliability across repeated trials. Its experiments found that even strong function-calling agents

could succeed on fewer than half of tasks in some settings, with substantial inconsistency across repeated attempts (Yao et al., 2024).

ToolSandbox similarly evaluates stateful, conversational tool use and emphasizes challenges such as state dependency, canonicalization, and insufficient information (Lu et al., 2025).

These approaches represent an important transition from evaluating isolated calls to evaluating **decision policies over trajectories**.

5.5 Realistic Live-API Evaluation

A significant recent development is evaluation against live or realistically executable API environments. Live API-Bench, published at EACL 2026, contains more than 2,500 invocable tools across 11 databases and evaluates error handling, sequential reasoning, parameter generation, response parsing, and robustness. Reported task-completion rates remained low across evaluated models, demonstrating the continuing difficulty of realistic multi-step tool calling (Elder et al., 2026).

Such benchmarks are particularly valuable because static API descriptions can conceal failures caused by changing schemas, heterogeneous interfaces, or execution-time constraints.

6. Proposed Evaluation Framework

A rigorous evaluation of tool-selection accuracy should report multiple complementary metrics.

6.1 Top-k Tool Selection

Report Top-1, Top-3, and Top-5 selection accuracy. This separates retrieval difficulty from final decision difficulty.

6.2 Necessity Accuracy

Treat "no tool required" as an explicit decision class:

$$Acc_{necessity} = \frac{TP_{tool} + TN_{no-tool}}{N}.$$

This captures both tool overuse and tool underuse.

6.3 Semantic Tool Accuracy

Instead of requiring an exact tool identifier, annotate equivalence classes of tools that satisfy the same functional objective under the task’s constraints. This is particularly important for redundant APIs.

6.4 Argument-Conditioned Selection Accuracy

A selected tool should count as fully correct only if its arguments are semantically valid. A hierarchical metric can therefore report:

$$P(\text{success}) = P(\text{selection}) \times P(\text{arguments} \mid \text{selection}) \times P(\text{execution} \mid \text{arguments}).$$

This decomposition provides a more useful engineering diagnosis than a single function-call accuracy score.

6.5 Sequence Selection Accuracy

For multi-tool tasks, evaluate:

- first-tool accuracy;
- per-step tool accuracy;
- valid-prefix length;
- sequence completion;
- functional equivalence of alternative sequences.

AgentBoard's emphasis on incremental progress provides a useful methodological precedent for such trajectory-level diagnostics (Ma et al., 2024).

6.6 Cost-Aware Utility

A theoretically correct tool may be operationally inferior. A useful metric should account for cost:

$$U = R_{\text{task}} - \lambda C_{\text{tool}} - \mu L_{\text{latency}} - \nu R_{\text{risk}},$$

where R_{task} is task reward, C_{tool} is monetary or computational cost, L_{latency} is latency, and R_{risk} represents safety or operational risk.

This reframes tool selection as **decision-making under constraints**, rather than simple classification.

6.7 Robustness to Tool-Set Scaling

Accuracy should be measured as the number of available tools increases. A model that achieves 95% accuracy with five tools but 60% with 500 tools exhibits a substantially different capability profile from a model that maintains stable performance.

Benchmark reports should therefore vary:

- number of tools;
- number of irrelevant tools;
- number of semantically similar tools;
- description length;
- schema complexity;
- tool freshness;
- tool reliability.

6.8 Repeated-Trial Consistency

Because tool selection is probabilistic, a single run can be misleading. τ -bench's pass^k methodology provides a useful model for evaluating reliability over repeated trials (Yao et al., 2024).

For tool selection, an analogous measure could report the probability that an agent selects a valid tool consistently across k independent executions.

7. Comparative Assessment of Existing Benchmarks

Benchmark / Approach	Main contribution	Multi-tool	Stateful	Selection-specific diagnosis	Realistic execution
Toolformer	Learned tool invocation	Limited	No	Partial	Limited
API-Bank	Tool planning, retrieval, calling	Yes	Limited	Stronger	Yes
ToolQA	Tool-dependent QA	Yes	Limited	Moderate	Yes
ToolBench / ToolLLM	Large-scale real APIs and retrieval	Yes	Limited	Strong	Yes
Gorilla / APIBench	Massive, changing APIs	Yes	Limited	Strong	Yes
AgentBench	General agent evaluation	Yes	Yes	Limited	Yes
WebArena	Realistic long-horizon web tasks	Yes	Yes	Limited	Strong
τ -bench	User-agent-tool interaction	Yes	Yes	Moderate	Strong
ToolSandbox	Stateful conversational tool use	Yes	Yes	Stronger	Strong
Live API-Bench	Large-scale live API evaluation	Yes	Yes	Strong	Very strong

This comparison reveals a persistent methodological trade-off. Benchmarks designed for **diagnosis** tend to use controlled tool sets and therefore provide clearer attribution, while benchmarks designed for **realism** emphasize long-horizon execution and functional outcomes but make individual selection errors harder to isolate.

A mature evaluation standard should combine both.

8. Research Challenges and Limitations

8.1 Absence of a Universal Ground Truth

Many tasks admit multiple valid tools and action sequences. Consequently, exact-match metrics can underestimate genuine competence. Evaluation must distinguish between canonical solutions and functionally equivalent alternatives.

8.2 Dynamic Tool Ecosystems

API documentation, schemas, authentication mechanisms, and tool availability change over time. Gorilla's retrieval-aware architecture demonstrates the importance of adapting to documentation changes (Patil et al., 2024).

A benchmark frozen at publication may therefore overestimate performance on real deployment workloads.

8.3 Tool Description Quality

Tool selection is partly a language-understanding problem over tool descriptions. Poorly written descriptions can create artificial difficulty, while overly informative descriptions can make selection trivial. Benchmarks need controlled variation in documentation quality.

8.4 Confounding Selection with Planning

In multi-step tasks, the "correct" next tool may depend on an internal plan. A benchmark that labels every tool independently can therefore mischaracterize strategic decisions. Tool selection should be evaluated both locally and in the context of the complete trajectory.

8.5 Failure Attribution

End-to-end success is attractive because it is easy to understand, but it is insufficient for scientific diagnosis. The evaluation literature increasingly argues for more holistic and trustworthy evaluation rather than relying on a small number of aggregate metrics (Litschko et al., 2023).

8.6 Safety and Policy Constraints

A technically capable agent may intentionally avoid a tool because the action violates policy or exceeds authorization. Therefore, "selection accuracy" cannot always be defined as choosing the tool that most directly satisfies the user request. Correct abstention can be the optimal behavior.

8.7 Benchmark Contamination and Synthetic Data

Several tool-use datasets rely on LLM-generated instructions or trajectories. ToolLLM, for example, uses ChatGPT in data construction (Qin et al., 2024). This improves scalability but introduces risks of synthetic-data artifacts, evaluator-model bias, and contamination.

9. Research Gaps and Future Directions

9.1 Tool Selection as Calibrated Decision-Making

Future systems should expose uncertainty over candidate tools rather than emitting only a single function call. Research should evaluate whether confidence estimates predict actual selection correctness and whether calibrated abstention improves safety and cost efficiency.

9.2 Causal Failure Attribution

Benchmarks should instrument trajectories so that researchers can distinguish:

task failure $\rightarrow$ $\left\{ \begin{array}{l} \text{wrong necessity} \\ \text{decision wrong tool} \\ \text{wrong arguments} \\ \text{wrong execution} \\ \text{interpretation wrong subse} \end{array} \right.$

This would transform tool-use benchmarks from leaderboard systems into diagnostic scientific instruments.

9.3 Dynamic and Continually Changing Tool Sets

Future benchmarks should modify tools after model deployment: schemas should change, tools should disappear, new tools should appear, and semantically equivalent services should compete. Such settings would directly test the adaptability emphasized by Gorilla and the realism pursued by Live API-Bench (Patil et al., 2024; Elder et al., 2026).

9.4 Adversarial and Ambiguous Tool Selection

Agents should be tested against misleading descriptions, redundant tools, conflicting documentation, incomplete schemas, and tools whose names are semantically deceptive. This would evaluate whether models genuinely understand tool capabilities rather than matching lexical patterns.

9.5 Learning from Tool Feedback

ToolReflection demonstrates a promising direction in which models learn from real-time API feedback and self-generated errors. Its experiments report substantial improvements on several out-of-distribution and multi-step tool-use settings (Polyakov et al., 2025).

Future work should investigate whether feedback-driven correction improves the **initial selection policy** or merely improves recovery after an incorrect choice.

9.6 Tool-Selection Efficiency

SMART demonstrates that excessive tool use can itself be a failure mode. Future evaluations should therefore report a Pareto frontier between task performance and tool expenditure rather than treating all successful trajectories as equivalent (Qian et al., 2025).

9.7 Human-Centered Evaluation

Real users care about whether an agent completes the task reliably, transparently, affordably, and safely—not merely whether its API call matches an annotation. Benchmarks such as τ -bench indicate the importance of incorporating user-agent interaction and domain-specific policies into evaluation (Yao et al., 2024).

9.8 Standardized Multi-Dimensional Reporting

The field would benefit from a reporting standard containing at least:

- Top-1 and Top-k tool-selection accuracy;
- tool-necessity accuracy;
- semantic-equivalence accuracy;
- argument validity;
- execution success;
- sequence-level success;
- average number of tool calls;
- latency and monetary cost;
- repeated-trial reliability;
- performance under tool-set scaling;
- robustness to schema/documentation changes;
- safety/policy violations.

Such reporting would make results across agent architectures more comparable.

10. Conclusion

Tool selection is emerging as a foundational capability of agentic AI, but current evaluation practices frequently subsume it under broader notions of function calling or end-to-end task success. The literature demonstrates substantial progress: ReAct established iterative reasoning and acting; Toolformer demonstrated learned API invocation; API-Bank introduced dedicated evaluation; ToolLLM and Gorilla scaled tool ecosystems; AgentBench and WebArena expanded evaluation into interactive environments; τ -bench and ToolSandbox introduced conversational and stateful interaction; and recent work has begun addressing tool overuse, self-correction, and live API evaluation.

Nevertheless, tool-selection accuracy remains insufficiently isolated as an evaluation target. The central methodological problem is that an agent's final success or failure can arise from multiple interacting causes. A rigorous evaluation framework must consequently distinguish necessity detection, tool identification, argument generation, execution, sequential planning, recovery, cost, and safety.

The most important conceptual shift is to treat tool selection not as a narrow function-name prediction problem but as **contextual decision-making under uncertainty over a dynamic action space**. This perspective naturally motivates semantic equivalence classes, calibrated abstention, cost-aware utility, trajectory-level analysis, repeated-trial reliability, and robustness to changing tool ecosystems.

As tool inventories continue to expand—from thousands of APIs to continuously evolving ecosystems of enterprise services and interoperable agent tools—the ability to select the right capability at the right time may become as important as the underlying language model's reasoning ability. Establishing rigorous, diagnostic, and reproducible measures of this capability is therefore essential for the scientific evaluation and safe deployment of multi-tool agentic AI.

References

1. Elder, Benjamin, Anupama Murthi, Jungkoo Kang, Ankita Rajaram Naik, Kiran Kate, Kinjal Basu, and Danish Contractor. (2026). *Live API-Bench: 2500+ Live APIs for Testing Multi-Step Tool Calling*. In **Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2026), Volume 1: Long Papers**, pp. 3092–3124. Association for Computational Linguistics. DOI: [10.18653/v1/2026.eacl-long.143](https://doi.org/10.18653/v1/2026.eacl-long.143).
2. Li, Minghao, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. (2023). *API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs*. In **Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)**, pp. 3102–3116. Association for Computational Linguistics. DOI: [10.18653/v1/2023.emnlp-main.187](https://doi.org/10.18653/v1/2023.emnlp-main.187). arXiv: [2304.08244](https://arxiv.org/abs/2304.08244).
3. Litschko, Robert, Max Müller-Eberstein, Rob van der Goot, Leon Weber-Genzel, and Barbara Plank. (2023). *Establishing Trustworthiness: Rethinking Tasks and Model Evaluation*. In **Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing**, pp. 193–203. Association for Computational Linguistics. DOI: [10.18653/v1/2023.emnlp-main.14](https://doi.org/10.18653/v1/2023.emnlp-main.14).
4. Lu, Jiarui, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. (2025). *ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities*. In **Findings of the Association for Computational Linguistics: NAACL 2025**, pp. 1160–1183. Association for Computational Linguistics. DOI: [10.18653/v1/2025.findings-naacl.65](https://doi.org/10.18653/v1/2025.findings-naacl.65). arXiv: [2408.04682](https://arxiv.org/abs/2408.04682).
5. Ma, Chang, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. (2024). *AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents*. arXiv preprint [arXiv:2401.13178](https://arxiv.org/abs/2401.13178).
6. Mishra, Venkatesh, Amir Saeidi, Satyam Raj, Mutsumi Nakamura, Gaowen Liu, Ali Payani, Jayanth Srinivasa, and Chitta Baral. (2025). *How Can Input Reformulation Improve Tool Usage Accuracy in a Complex Dynamic Environment? A Study on tau-bench*. In **Findings of the Association for Computational Linguistics: EMNLP 2025**, pp. 22949–22972. Association for Computational Linguistics. DOI: [10.18653/v1/2025.findings-emnlp.1250](https://doi.org/10.18653/v1/2025.findings-emnlp.1250).
7. Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. (2024). *Gorilla: Large Language Model Connected with Massive APIs*. In **Advances in Neural Information Processing Systems 37 (NeurIPS 2024)**, pp. 126544–126565. DOI: [10.52202/079017-4020](https://doi.org/10.52202/079017-4020). arXiv: [2305.15334](https://arxiv.org/abs/2305.15334).

8. Polyakov, Gregory, Ilseyar Alimova, Dmitry Abulkhanov, Ivan Sedykh, Andrey Bout, Sergey Nikolenko, and Irina Piontkovskaya. (2025). *ToolReflection: Improving Large Language Models for Real-World API Calls with Self-Generated Data*. In **Proceedings of the 1st Workshop for Research on Agent Language Models (REALM 2025)**, pp. 184–199. Association for Computational Linguistics. DOI: [10.18653/v1/2025.realm-1.14](https://doi.org/10.18653/v1/2025.realm-1.14).
9. Qian, Cheng, Emre Can Acikgoz, Hongru Wang, Xiusi Chen, Avirup Sil, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. (2025). *SMART: Self-Aware Agent for Tool Overuse Mitigation*. arXiv preprint [arXiv:2502.11435](https://arxiv.org/abs/2502.11435).
10. Qin, Yujia, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. (2024). *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs*. In **International Conference on Learning Representations (ICLR 2024)**. arXiv: [2307.16789](https://arxiv.org/abs/2307.16789).
11. Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. In **Advances in Neural Information Processing Systems 36 (NeurIPS 2023)**. arXiv: [2302.04761](https://arxiv.org/abs/2302.04761).
12. Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. In **International Conference on Learning Representations (ICLR 2023)**. arXiv: [2210.03629](https://arxiv.org/abs/2210.03629). DOI: [10.48550/arXiv.2210.03629](https://doi.org/10.48550/arXiv.2210.03629).
13. Yao, Shunyu, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. (2024). *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. arXiv preprint [arXiv:2406.12045](https://arxiv.org/abs/2406.12045).
14. Zhou, Shuyan, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. (2024). *WebArena: A Realistic Web Environment for Building Autonomous Agents*. In **International Conference on Learning Representations (ICLR 2024)**. arXiv: [2307.13854](https://arxiv.org/abs/2307.13854).
15. Zhuang, Yuchen, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. (2023). *ToolQA: A Dataset for LLM Question Answering with External Tools*. In **Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track**, pp. 50117–50143. DOI: [10.52202/075280-2180](https://doi.org/10.52202/075280-2180). arXiv: [2306.13304](https://arxiv.org/abs/2306.13304).
16. Liu, Xiao, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. (2024). *AgentBench: Evaluating LLMs as Agents*. In **International Conference on Learning Representations (ICLR 2024)**. arXiv: [2308.03688](https://arxiv.org/abs/2308.03688).